



Security Audit Report

Valdora Eth Vault

v1.0

July 27, 2026

Table of Contents

Table of Contents

License

Disclaimer

Introduction

Purpose of This Report

Codebase Submitted for the Audit

Methodology

Functionality Overview

How to Read This Report

Code Quality Criteria

Summary of Findings

Detailed Findings

1. Floored-integer override NAV underpays redeemers, causing realized loss of funds
2. Pre-seeded AUM lets the first depositor capture value through 1:1 share minting
3. Just-in-time deposits can harvest addReward yield via instant internal redemption
4. Redemption locks request-time NAV, so oracle updates redistribute holder value
5. maxSupply at or below MINIMUM_LIQUIDITY permanently bricks the first deposit
6. Override deviation guard skipped at zero NAV allows arbitrary settlement
7. Admin can reduce oracle AUM guardrails to no-ops, defeating price bounds
8. Rebasing or interest-bearing assets are mis-accounted, bricking redemptions
9. Outbound fee tokens can make external vault accounting overstate transferred assets
10. Queued redemptions lack user cancellation, locking burned-share value indefinitely
11. Rejected redemptions can re-mint shares above the maxSupply ceiling
12. ExternallyManaged vaults cannot recover directly-sent assets
13. Pausing does not freeze share-token transfers, leaving claims movable
14. Floored-integer NAV and inconsistent empty-vault NAV mislead integrators
15. symbol_emptiness is not validated, degrading off-chain display
16. Unused custom errors increase code surface without affecting behavior

Proof of Concept

License



THIS WORK IS LICENSED UNDER A [CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE](https://creativecommons.org/licenses/by-nc/4.0/).

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED "AS IS", WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUCT ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

<https://oaksecurity.io>

info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by TBC to perform a security audit of the Valdora Eth Vault Contracts.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	URL	Type	Commit
evm-vaults	https://github.com/Liquid-Zig/evm-vaults	Full	f394dc66da4fe3b8e97d8306535a64dea99b7027

Scope	<pre>src/ errors/ VaultErrors.sol interfaces/ IVaultEvents.sol libraries/ NavMath.sol types/ VaultTypes.sol Vault.sol</pre>
-------	---

Fixes Verified at Commit

da8e2d21c40a78f4693a4ac67cd0c774c531ddc1

Note that only fixes to the issues described in this report have been reviewed at this commit.

Any further changes such as additional features have not been reviewed.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
 - a. Race condition analysis
 - b. Under-/overflow issues
 - c. Key management vulnerabilities
4. Report preparation.

Functionality Overview

The repository contains a single UUPS-upgradeable ERC20 vault contract (`Vault.sol`) built with Foundry, along with supporting libraries, types, interfaces, and error definitions. Users deposit a single ERC20 asset (such as USDC) and receive vault shares minted proportional to their contribution. Net Asset Value (NAV) is computed as AUM divided by total supply, with share-to-asset conversions handled via full-precision `mulDiv` arithmetic in `NavMath.sol`. A minimum liquidity lock of 1000 shares to the dead address mitigates inflation-style attacks on deposit.

The vault operates in one of two modes set at initialization: `ExternallyManaged`, where deposited assets are forwarded to a designated funds manager and redemptions are queued for approval, or `InternallyManaged`, where assets are held in the contract and redemptions settle immediately. Oracle-based AUM updates are rate-limited by a basis-point cap (default 2%) and a cooldown period (default 24 hours). Role-based access controls govern AUM updates, redemption approvals, and reward top-ups, while a two-step admin transfer pattern protects upgrade authorization.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or losses, with medium to high probability.
Minor	A violation of best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects such as user experience or readability. This category may also include opinionated recommendations that the project team might not agree with.

The **status** of an issue can be one of the following: **Resolved**, **Acknowledged**, or **Partially Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	Medium	Simple functions, broad dual-mode feature surface
Code readability and clarity	High	–
Level of documentation	High	Exhaustive NatSpec, rationale comments, README, docs
Test coverage	High	Near-total lines/functions

Summary of Findings

No	Description	Severity	Status
1	Floored-integer override NAV underpays redeemers, causing realized loss of funds	Major	Resolved
2	Pre-seeded AUM lets the first depositor capture value through 1:1 share minting	Minor	Resolved
3	Just-in-time deposits can harvest addReward yield via instant internal redemption	Minor	Resolved
4	Redemption locks request-time NAV, so oracle updates redistribute holder value	Minor	Acknowledged
5	maxSupply at or below MINIMUM_LIQUIDITY permanently bricks the first deposit	Minor	Resolved
6	Override deviation guard skipped at zero NAV allows arbitrary settlement	Minor	Resolved
7	Admin can reduce oracle AUM guardrails to no-ops, defeating price bounds	Minor	Resolved
8	Rebasing or interest-bearing assets are mis-accounted, bricking redemptions	Minor	Acknowledged
9	Outbound fee tokens can make external vault accounting overstate transferred assets	Minor	Resolved
10	Queued redemptions lack user cancellation, locking burned-share value indefinitely	Minor	Resolved
11	Rejected redemptions can re-mint shares above the maxSupply ceiling	Minor	Resolved
12	ExternallyManaged vaults cannot recover directly-sent assets	Informational	Resolved

No	Description	Severity	Status
13	Pausing does not freeze share-token transfers, leaving claims movable	Informational	Resolved
14	Floored-integer NAV and inconsistent empty-vault NAV mislead integrators	Informational	Resolved
15	symbol_emptiness is not validated, degrading off-chain display	Informational	Resolved
16	Unused custom errors increase code surface without affecting behavior	Informational	Resolved

Detailed Findings

1. Floored-integer override NAV underpays redeemers, causing realized loss of funds

Severity: Major

- `src/Vault.sol:417`
- `src/Vault.sol:403-421`
- `src/Vault.sol:317, 320`
- `src/libraries/NavMath.sol:20-25, 57-66`

In `src/Vault.sol:417`, the `approveRequest` function recomputes the override payout as the raw product `req.assetAmount = req.shares * overrideNav`, unlike the default path, which prices with full-precision `Math.mulDiv`. In lines 406-416, `overrideNav` is bounded to within `overrideNavMaxDeviationBps` of `req.originalNav`, and in line 317, `req.originalNav` is the request-time NAV floored to an integer (`navFrom = aum / supply`). Because the share token mirrors the asset decimals and the first deposit mints one-to-one, per-share NAV sits near 1, so the only integer inside the default 200 bps band is `floor(NAV)` itself.

The manager is therefore forced to settle at `req.shares * floor(NAV)` rather than the correct `floor(req.shares * NAV)`; applying an override even at `overrideNav == req.originalNav` discards the fractional NAV and underpays. The shares are already burned in `requestRedeem` (line 326), so the loss is realized. The POC (see below) shows a routine 2% appreciation paying `300_000_000` where the correct amount is `306_000_000` (a 2% loss); at a true NAV of 1.9 the forced `overrideNav == 1` is an approximately 47% loss.

As a result, every override settlement on a fractional NAV underpays the redeemer by the fractional part of the NAV. The documented settlement lever cannot express any non-integer NAV, so an honest manager using it as documented silently mis-settles redemptions with realized, unrecoverable loss.

The defect manifests under normal, expected use of the documented override path by an honest `REDEEM_MANAGER_ROLE` holder and requires no special or malicious privileged action. We classify this issue as major since it produces incorrect payouts and realized loss of user funds through the documented settlement path.

→ **Proof of Concept: P.1**

Recommendation

We recommend pricing the override with full precision: carry `overrideNav` at a fixed scale (for example `1e18` or `10 ** decimals`) and compute `Math.mulDiv(req.shares, overrideNav, SCALE)` with `req.originalNav` derived at the same scale, or have the manager supply a corrected AUM

and settle `Math.mulDiv(req.shares, overrideAum, supply)`, so the deviation check and the payout share one precise scale.

Status: Resolved

2. Pre-seeded AUM lets the first depositor capture value through 1:1 share minting

Severity: Minor

- `src/Vault.sol:267-306` (`deposit` - first deposit mints shares 1:1 even when `latestAum` is non-zero)
- `src/Vault.sol:321-379` (`requestRedeem` - values shares against pre-seeded AUM plus the deposit)
- `src/Vault.sol:550-566` (`addReward` - can credit `latestAum` while `totalSupply` is zero)
- `src/Vault.sol:705-718` (`sweepUnaccountedFunds` - can credit donated assets to `latestAum` while `totalSupply` is zero)
- `src/Vault.sol:513-542` (`updateAum` - can set `latestAum` while `totalSupply` is zero)
- `src/libraries/NavMath.sol:36-48` (`sharesFrom` - returns `depositAmount` when supply is zero)

The vault can enter a state where `latestAum` is positive while `totalSupply` is zero. In `InternallyManaged` mode, this can happen through `addReward` or `sweepUnaccountedFunds`. In `ExternallyManaged` mode, this can happen through `updateAum`.

When the first user deposits after AUM has already been credited, the deposit is still priced 1:1 because `sharesFrom` ignores AUM whenever supply is zero.

The first depositor therefore receives shares as if the vault were empty, but those shares are immediately valued against both the user's deposit and the pre-seeded AUM. The first-deposit dead shares reduce the amount captured by the depositor, but they do not remove the issue. They only leave `MINIMUM_LIQUIDITY` shares worth of value assigned to the dead address.

In `InternallyManaged` mode, the issue is directly exploitable because `requestRedeem` settles immediately from the vault's asset balance. In `ExternallyManaged` mode, the first depositor can create an inflated redemption claim, but cannot force final payment without the redeem manager approving and funding the request.

Consider the following scenario in an `InternallyManaged` vault:

1. A fresh vault has `latestAum = 0` and `totalSupply = 0`.

2. An account with `ADD_REWARD_ROLE` calls `addReward` with `1,000,000` assets before any user deposits.
3. The vault now has `latestAum = 1,000,000` and `totalSupply = 0`.
4. Alice is the first depositor and deposits `2,000` assets.
5. `sharesFrom` returns `2,000` shares because `totalSupply` is zero.
6. The vault mints `1,000` dead shares and `1,000` shares to Alice.
7. After the deposit, `latestAum = 1,002,000` and `totalSupply = 2,000`.
8. Each share is worth `501` assets.
9. Alice calls `requestRedeem` for her `1,000` shares.

Because the vault is `InternallyManaged`, the redemption settles instantly:

$$1,000 * 1,002,000 / 2,000 = 501,000$$

Alice deposited `2,000` assets and receives `501,000` assets, extracting `499,000` assets from the pre-seeded reward. The remaining `501,000` assets of value belong to the dead shares and are permanently locked.

The same issue can arise if assets are donated to an empty `InternallyManaged` vault and then credited through `sweepUnaccountedFunds`.

Consider also the following scenario in the `ExternallyManaged` variant:

1. A fresh external vault has `latestAum = 0` and `totalSupply = 0`.
2. The AUM updater calls `updateAum` with `1,000,000` before the first deposit.
3. Alice deposits `1,000,000` assets as the first depositor.
4. The vault mints `1,000` dead shares and `999,000` shares to Alice.
5. After the deposit, `latestAum = 2,000,000` and `totalSupply = 1,000,000`.
6. Alice requests to redeem her `999,000` shares.

The redemption request is valued as:

$$999,000 * 2,000,000 / 1,000,000 = 1,998,000$$

Alice cannot force the payout in `ExternallyManaged` mode because `approveRequest` requires `REDEEM_MANAGER_ROLE` and the redeem manager must fund the request. However, if the redeem manager treats the request as valid and approves it, the off-chain managed pool pays the inflated claim.

Recommendation

We recommend preventing positive AUM from existing while `totalSupply` is zero: in `addReward`, `sweepUnaccountedFunds`, and `updateAum`, revert when the vault would credit or set a positive `latestAum` while `totalSupply` is zero, so AUM can never be seeded ahead of the first deposit.

As defense in depth, also fix `sharesFrom` itself, which currently returns `depositAmount` one-to-one whenever `supply == 0` and ignores `aum`. Handle the `supply == 0 && aum != 0` case explicitly by either pricing the first deposit against the seeded AUM or reverting, so a seeded balance cannot be captured 1:1 even if one of the entrypoint guards is missed.

Status: Resolved

3. Just-in-time deposits can harvest `addReward` yield via instant internal redemption

Severity: Minor

- `src/Vault.sol:267-284` (`deposit` - shares minted at pre-reward NAV, then AUM credited)
- `src/Vault.sol:321-379` (`requestRedeem` - InternallyManaged redemptions settle immediately)
- `src/Vault.sol:550-566` (`addReward` - reward is credited directly to `latestAum`)
- `src/libraries/NavMath.sol:36-48` (`sharesFrom` - deposit share mint formula)
- `src/libraries/NavMath.sol:57-67` (`assetForShares` - redemption payout formula)

In an `InternallyManaged` vault, `addReward` pulls assets into the vault and immediately increases `latestAum` without minting shares.

This atomically increases NAV for all currently outstanding shares. Because deposits are permissionless and are priced against the pre-deposit AUM/supply, a user can enter before a known, scheduled, or otherwise observable reward transaction and receive shares at the old NAV.

The same user can then redeem immediately after the reward. In `InternallyManaged` mode, `requestRedeem` computes the payout from the current `latestAum` and current supply, burns the shares, and transfers the assets in the same call.

There is no holding period, redemption delay, same-block deposit/redeem guard, or reward streaming mechanism. As a result, a just-in-time depositor can capture a pro-rata slice of a discrete reward despite not bearing the position risk before the reward was added.

Consider the following scenario in an `InternallyManaged` vault:

1. Existing vault state is $AUM = 1,000,000$ assets and $supply = 1,000,000$ shares, so the NAV is 1 asset per share.
2. A role-gated call to `addReward` with reward amount $100,000$ assets is expected or has already been submitted.
3. The attacker submits a deposit of $1,000,000$ assets ordered ahead of the reward, receiving $1,000,000$ shares at the pre-reward NAV.
4. The reward transaction executes and sets $latestAum = 2,100,000$.
5. The attacker then calls `requestRedeem` for their shares, which settles instantly from the contract balance.

The attacker owns $1,000,000$ of the $2,000,000$ total shares after the deposit, so the instant redemption pays:

$$1,000,000 / 2,000,000 * 2,100,000 = 1,050,000$$

The attacker recovers the $1,000,000$ asset deposit plus $50,000$ assets from the reward. Existing holders receive only the remaining $50,000$ assets of the $100,000$ asset reward, even though they held all shares before the reward transaction.

Recommendation

We recommend avoiding crediting discrete rewards directly to all immediately redeemable shares. Options include:

1. Stream rewards into AUM over time instead of applying the full amount atomically.
2. Add a minimum holding period or redemption delay for newly minted shares.
3. Prevent same-block or short-window deposit-then-redeem flows from receiving newly added rewards.
4. Snapshot eligible supply before `addReward` and distribute the reward only to shares that existed before the reward was added.

Status: Resolved

4. Redemption locks request-time NAV, so oracle updates redistribute holder value

Severity: Minor

- `src/Vault.sol:317-354, 400-424, 460-482` (`requestRedeem`, `approveRequest`, `rejectRequest`)
- `src/Vault.sol:253-270` (`deposit`), `499-529` (`updateAum`; overwrite at line 523, bound at 518)

In `src/Vault.sol:320` and `330`, `requestRedeem` locks the payout at the request-time NAV (`mulDiv(shares, latestAum, supply)`) and debits `latestAum` by it; in line `270`, `deposit` credits `latestAum`; and in line `523`, `updateAum` overwrites `latestAum` absolutely, measuring its change bound (line `518`) against the current, mutable value. The reservation and the absolute mark are never reconciled, so any flow that interleaves with an oracle update desynchronizes from the live pool.

Privileged paths: `rejectRequest` (`460-482`) re-credits the stale `req.assetAmount` and re-mints the shares, so the rejected user re-enters at the old ratio; `approveRequest` pays the stale reservation, and its override is capped at 10% (`overrideNavMaxDeviationBps`), so a larger drawdown forces overpayment at the stale higher NAV.

Unprivileged paths: because `updateAum` overwrites absolutely and bounds against the live `latestAum`, a `requestRedeem` ordered before a pending update prices at the stale NAV and shifts the baseline. With `latestAum = supply = 1,000,000` and a pending `updateAum(980,000)` (a 2% markdown), redeeming `20,000` shares first locks a `20,000` claim and sets `latestAum = 980,000`; the update then computes `0` bps, dropping the markdown and leaving a pre-drawdown claim (`20,000` versus the fair `19,600`). A deposit landing before a stale update behaves the same way: it is either omitted from `latestAum` or reverts `AumChangeTooLarge`, briefly blocking oracle updates.

As a result, a redemption or deposit that interleaves with an oracle mark redistributes per-share value between the affected user and the remaining holders, or briefly blocks a valid `updateAum`. Each event is bounded by the per-call AUM change limit, total claims are conserved, and payouts are funded off-chain by the manager via `_pullExact` rather than from the pool, so this is a bounded redistribution and a retrievable update block, not a protocol leak; it self-corrects on the next mark.

We classify this issue as minor since the per-event magnitude is bounded by the oracle rate limit, claims are conserved with no principal at risk and self-correct on the next `updateAum`, and the deposit-side revert only delays a privileged update that can be retried.

Recommendation

We recommend reconciling the request-time reservation with the absolute oracle mark. Settle and reject in economically-neutral terms: re-mint the shares the reserved `req.assetAmount` buys at the current NAV, or re-derive the approval payout from live `latestAum` and supply with the override as a bounded adjustment. Also make `updateAum` robust to an intervening user flow by gating it on the expected `latestAum`, `totalSupply`, and `pendingRedemptionAum` (or an accounting nonce), reverting if they changed.

Status: Acknowledged

The client acknowledges the issue stating that the redistribution is bounded by the AUM change rate limit, which has since been further constrained by a 50% hard ceiling and a non-zero cooldown introduced in a follow-up commit. Claims are conserved because `requestRedeem` and `rejectRequest` debit and re-credit `latestAum` in lockstep, and externally managed payouts are funded by the redeem manager rather than drawn from shared pool assets. The client accepts the deposit-side revert as a retrievable delay with no fund loss, and considers adding expected-state gating to `updateAum` disproportionate given the minor, self-correcting nature of the effect.

5. `maxSupply` at or below `MINIMUM_LIQUIDITY` permanently bricks the first deposit

Severity: Minor

- `src/Vault.sol:197, 263-284, 684-689`

In `src/Vault.sol:263` and `278-279`, the deposit function requires a first deposit to pass both `shares <= maxSupply` (with `preSupply == 0`) and `shares > MINIMUM_LIQUIDITY` (where `MINIMUM_LIQUIDITY` is `1000`), and `shares` equals `amount` because `sharesFrom` returns one-to-one at zero supply. If `maxSupply` is in the range 1 to `1000`, no value threads both gates: `1000` shares revert `FirstDepositBelowMinimum` and `1001` revert `ExceedsMaxSupply`. Neither `initialize` (`src/Vault.sol:197`) nor `updateMaxSupply` (`src/Vault.sol:685`) validates `maxSupply` against `MINIMUM_LIQUIDITY`.

As a result, a vault initialized, or reconfigured while supply is zero, with `maxSupply` in 1 to `1000` can never accept its first deposit and is permanently non-functional. No deposited funds are at risk, and the admin can recover by raising the cap.

We classify this issue as minor since it is a self-inflicted denial of service from missing input validation, reachable only by the deployer or admin, with no third-party funds at risk.

Recommendation

We recommend requiring `maxSupply` to be strictly greater than `MINIMUM_LIQUIDITY` in `initialize` and `updateMaxSupply`, reverting with a dedicated error, or alternatively evaluating `FirstDepositBelowMinimum` before `ExceedsMaxSupply` so the root-cause error surfaces.

Status: Resolved

6. Override deviation guard skipped at zero NAV allows arbitrary settlement

Severity: Minor

- `src/Vault.sol:406`
- `src/Vault.sol:417-421`
- `src/libraries/NavMath.sol:20-25`

In `src/Vault.sol:406`, the `approveRequest` function guards the override deviation check with `if (req.originalNav != 0)`. When `originalNav` is `0`, the basis-point bound is skipped entirely and `req.assetAmount = req.shares * overrideNav` is accepted with no ceiling, rejected only if it computes to exactly `0` (line 418). `originalNav` is the floored `navFrom(aum, supply)`, which returns `0` whenever `aum < supply`. That below-par state is reachable in `ExternallyManaged` mode through one allowed 2% `updateAum` downtick: with `aum == supply` a single downtick makes `aum < supply`, so `originalNav` floors to `0` while `assetForShares` stays strictly positive and the request is created. The POC settles an override of `100` (roughly `1_000_000` bps deviation) with no revert.

As a result, a bound designed to constrain redeem-manager discretion fails open in exactly the impaired state where it matters. The payout is pulled from the manager and forwarded to the user, so the harm is confined to a single redeeming user rather than the remaining holders.

We classify this issue as minor since reaching the below-par state requires `AUM_UPDATER_ROLE`, executing the override requires `REDEEM_MANAGER_ROLE`, and the harm is confined to a single redeemer whose shares are already burned.

→ **Proof of Concept: P.2**

Recommendation

We recommend not skipping the deviation check when `originalNav == 0`: carry NAV at higher precision so `originalNav` is zero only for a genuinely empty vault, or bound the override against the full-precision request-time `assetAmount`. If a zero reference is legitimately possible, enforce

an absolute bound rather than skipping the check.

Status: Resolved

7. Admin can reduce oracle AUM guardrails to no-ops, defeating price bounds

Severity: Minor

- `src/Vault.sol:650-657, 662-666`
- `src/Vault.sol:505-521`

In `src/Vault.sol:650-651`, the `updateAumChangeLimit` function accepts any value up to `BPS_DENOMINATOR` (100%) with no meaningful ceiling, so a single `updateAum` can double or zero the AUM, reducing the per-call change limit (default 200 bps), presented as a guard against bad oracle data, to a no-op.

In `src/Vault.sol:662-666`, the `updateAumUpdateCooldown` function accepts `0` with no lower bound; with a zero cooldown `nextAllowed == lastAumUpdateAt`, so the check in line 508 always passes and updates can be chained within one block, compounding the per-call limit. The setter also has no upper bound: in line 507, the unchecked addition `lastAumUpdateAt + aumUpdateCooldownSecs` overflows for a value near the `uint64` maximum and reverts, freezing `updateAum` until the admin resets the cooldown; an overflow-safe form such as `block.timestamp - lastAumUpdateAt < aumUpdateCooldownSecs` avoids this. Even at defaults, the cooldown only spaces updates and caps no cumulative drift.

Impact: the oracle bounds that ostensibly protect the share price can be turned off, compounded within a block, or frozen entirely, so the NAV, and thus every deposit and redemption price, can be moved without effective on-chain limit, or updates can be disabled.

We classify this issue as minor since it reflects the trust already placed in the oracle and admin roles and is admin-recoverable, which falls under the centralization guidance.

Recommendation

We recommend enforcing sane bounds on the configurable guardrails: a maximum on `aumChangeLimitBps` well below 100%, and both a non-zero minimum and a sane maximum on `aumUpdateCooldownSecs`; computing the cooldown comparison in an overflow-safe form such as `block.timestamp - lastAumUpdateAt < aumUpdateCooldownSecs`; and considering a cumulative-drift cap over a rolling window.

Status: Resolved

8. Rebasing or interest-bearing assets are mis-accounted, bricking redemptions

Severity: Minor

- `src/Vault.sol:854-859, 357-363, 695-705`

In `src/Vault.sol:854-859`, the `_pullExact` function reconciles booked `latestAum` against `asset.balanceOf(address(this))` and rejects any transfer whose measured delta differs from the requested amount, barring fee-on-transfer and deflationary tokens, but it gives no protection against rebasing or interest-bearing assets (for example `aTokens`, `stETH`, or elastic-supply tokens) whose `balanceOf` changes out of band. In `InternallyManaged` mode a positive rebase accrues surplus invisible until an admin `sweepUnaccountedFunds`, and a negative rebase drops the balance below `latestAum`, so in line 358 instant redemptions revert `InsufficientFunds` while `latestAum` still claims the value exists.

As a result, for a deployment configured with a rebasing or interest-bearing asset, positive-rebase yield accrues to no one until an admin sweep, and a negative rebase leaves booked AUM above the real balance, over-valuing shares and bricking `InternallyManaged` redemptions. Fixed-balance assets such as USDC are unaffected.

The `solidity.md` file documents the fee-on-transfer rejection ("reject fee-on-transfer by checking `received == amount`"), but neither `solidity.md` nor `README.md` addresses rebasing or interest-bearing assets.

We classify this issue as minor since it is an unhandled asset assumption that depends on the deployed asset rather than a flaw currently exploitable with the intended USDC-style asset.

Recommendation

We recommend documenting that the asset must be non-rebasing and non-interest-bearing and validating or constraining the asset at initialization; if such assets must be supported, deriving holdings from the live balance (net of pending obligations) rather than an independently-booked `latestAum`.

Status: Acknowledged

The client acknowledges the issue stating that the vault is intentionally designed for fixed-balance assets such as USDC, and rebasing or interest-bearing assets are considered out of scope. In response to the recommendation, the non-rebasing and non-interest-bearing requirement has been documented in the Vault contract header, the README, and the

solidity.md file. The client notes that reliable on-chain detection of rebasing behavior is not feasible, so the constraint is enforced through documentation and deployment policy rather than a runtime check.

9. Outbound fee tokens can make external vault accounting overstate transferred assets

Severity: Minor

- `src/Vault.sol:267-306` (`deposit` - exact-checks the inbound pull, then forwards assets to `fundsManager`)
- `src/Vault.sol:400-452` (`approveRequest` - exact-checks the manager pull, then forwards the payout to the user)
- `src/Vault.sol:863-872` (`_pullExact` - rejects inbound fee-on-transfer behavior only)

The `Vault` contract rejects fee-on-transfer behavior when it pulls assets into the vault. The `_pullExact` helper measures the vault's balance before and after `safeTransferFrom` and reverts unless the vault receives the exact requested amount.

This protects inbound transfers, but it does not protect outbound transfers. In `ExternallyManaged` deposits, the vault first pulls the full deposit amount from the user, mints shares, credits `latestAum`, and then forwards the same amount to `fundsManager`. This calls `SafeERC20.safeTransfer` checks that the token call succeeds, but it does not verify how many tokens the recipient actually receives. A token that charges a fee only when the vault transfers tokens out can therefore pass the inbound exact-transfer check while short-paying the `fundsManager`.

The same pattern exists in `approveRequest`. The vault pulls the exact booked payout from the redeem manager, but then forwards the payout to the user without measuring the user's received balance:

```
_pullExact(msg.sender, payout);  
asset.safeTransfer(req.user, payout);
```

The request is marked fulfilled before the outbound transfer. Therefore, for outbound fee tokens, the vault can emit and account for a full deposit or redemption payout even though the final recipient receives less.

Recommendation

We recommend measuring recipient balance deltas for outbound transfers that are expected to deliver an exact amount.

- For `ExternallyManaged` deposits, check the `fundsManager` balance before and after forwarding the deposit:
- For `approveRequest`, apply the same pattern to the user's balance before and after the payout transfer.

Status: Resolved

10. Queued redemptions lack user cancellation, locking burned-share value indefinitely

Severity: Minor

- `src/Vault.sol:307-367, 386-394, 460-468`
- `src/Vault.sol:641-645`

In `src/Vault.sol:326`, the `requestRedeem` function burns the user's shares immediately and records a Pending request in `ExternallyManaged` mode, but transfers no assets; the payout is made only later by a `REDEEM_MANAGER_ROLE` holder through `approveRequest`. The only other exit, `rejectRequest`, is also restricted to `REDEEM_MANAGER_ROLE`, and both are `whenNotPaused`. There is no user-callable cancellation and no timeout, so once shares are burned the user has no self-service recourse: a manager who never acts, a lost key, an indefinite pause, or a revoked role leaves the request Pending forever. In `src/Vault.sol:641-645`, `revokeRole` enforces no minimum member count, so the admin can empty `REDEEM_MANAGER_ROLE` entirely while requests are outstanding.

As a result, in-flight redemption value stays locked with no on-chain user recovery for as long as no privileged party acts. The admin can recover it by re-granting `REDEEM_MANAGER_ROLE`, unpausing, or upgrading the contract, so the lock is not permanent under normal conditions; it becomes genuinely unrecoverable only in the compound case where the admin is also unavailable or malicious. In all cases the user has no self-service exit.

We classify this issue as minor since the lock does not arise under honest operation and depends on redeem-manager inaction, key loss, role revocation, or an indefinite admin pause, which is a privileged-role and centralization liveness concern.

Recommendation

We recommend adding a user-callable cancel path for a caller's own Pending request that mirrors `rejectRequest` accounting (re-credit `latestAum`, subtract from `pendingRedemptionAum`, re-mint the burned shares, mark a terminal cancelled state), optionally behind a timeout, and

preventing `revokeRole` from emptying `REDEEM_MANAGER_ROLE` while `pendingRequestCount` is non-zero.

Status: Resolved

11. Rejected redemptions can re-mint shares above the `maxSupply` ceiling

Severity: Minor

- `src/Vault.sol:267-279` (`deposit` - enforces the live `maxSupply` ceiling)
- `src/Vault.sol:474-496` (`rejectRequest` - re-mints burned shares without checking `maxSupply`)
- `src/Vault.sol:694-696` (`updateMaxSupply` - documents `maxSupply` as a live total-supply ceiling)

The Vault contract documents `maxSupply` as a live `totalSupply` ceiling and enforces that ceiling when users deposit. In `deposit`, the vault computes the shares to mint and reverts if minting them would push live supply above the cap.

However, in `ExternallyManaged` mode, `requestRedeem` burns shares immediately while the redemption request remains pending. This temporarily reduces `totalSupply` and frees capacity under `maxSupply`. Other users can then deposit into that freed capacity.

If the redeem manager later rejects the original redemption request, `rejectRequest` re-mints the burned shares without checking whether the cap has already been refilled:

```
req.status = RequestStatus.Rejected;
latestAum += req.assetAmount;
pendingRedemptionAum -= req.assetAmount;
_pendingRequestIds.remove(id);

_mint(req.user, req.shares);
```

As a result, a rejected redemption can push live `totalSupply` above `maxSupply`. The code comment in `rejectRequest` states that no supply-cap raise is needed because the function restores previously valid supply, but that assumption does not account for intervening deposits while the request is pending.

Consider the following scenario in an `ExternallyManaged` vault:

1. The admin sets `maxSupply = 1,000,000` shares.
2. Alice deposits until `totalSupply = 1,000,000`, filling the cap.

3. Alice requests to redeem 500,000 shares. The request burns those shares immediately, so `totalSupply = 500,000`.
4. Bob deposits and mints 500,000 shares, bringing `totalSupply` back to 1,000,000.
5. The redeem manager rejects Alice's redemption request.
6. `rejectRequest` re-mints Alice's 500,000 burned shares, so `totalSupply = 1,500,000` while `maxSupply` remains 1,000,000.

Recommendation

We recommend reserving pending redemption shares against the supply cap instead of treating burned pending shares as reusable capacity. For example, track `pendingRedemptionShares` in addition to `pendingRedemptionAum`:

1. Increase `pendingRedemptionShares` by `shares` when `requestRedeem` creates a pending request in `ExternallyManaged` mode.
2. Decrease `pendingRedemptionShares` by `req.shares` when the request is approved (`approveRequest`) or rejected (`rejectRequest`), mirroring the existing `pendingRedemptionAum` release.
3. In deposit, enforce the cap against effective supply:

```
uint256 effectiveSupply = totalSupply() + pendingRedemptionShares;
if (effectiveSupply + shares > maxSupply) {
    revert VaultErrors.ExceedsMaxSupply(effectiveSupply, maxSupply, shares);
}
```

This preserves enough headroom for rejected requests to be re-minted without exceeding the cap.

Status: Resolved

12. ExternallyManaged vaults cannot recover directly-sent assets

Severity: Informational

- `src/Vault.sol:695-705, 288-290, 434-438`

In `src/Vault.sol:288-290`, the `deposit` function forwards all deposited assets to `fundsManager` in `ExternallyManaged` mode, and redemptions are paid from the redeem manager's own balance, so the vault holds approximately zero asset balance. In `src/Vault.sol:696-698`,

`sweepUnaccountedFunds` is hard-gated to `InternallyManaged` only and there is no other rescue function, so any asset sent directly to an `ExternallyManaged` vault, by user error or dust, can never leave.

As a result, any asset sent directly to an `ExternallyManaged` vault has no in-protocol recovery path. This never affects depositor funds or solvency as pricing reads `latestAum`, not the token balance, so it is confined to assets misdirected outside the intended flow: realistically dust, airdrops, or a user transferring the token to the vault address instead of calling `deposit`. The amount is bounded only by that self-inflicted mistake, and can only lock the sender's own tokens. The admin can add a rescue via a UUPS upgrade, so recovery requires shipping new logic rather than an existing function.

Recommendation

We recommend providing an admin-only rescue path for the asset in `ExternallyManaged` mode that transfers the contract's asset balance to a designated address, while keeping the `InternallyManaged` sweep semantics of crediting surplus to AUM.

Status: Resolved

13. Pausing does not freeze share-token transfers, leaving claims movable

Severity: Informational

- `src/Vault.sol:123, 145-148, 561-573`

In `src/Vault.sol:145-148`, the `whenNotPaused` modifier guards `deposit`, `requestRedeem`, `approveRequest`, `rejectRequest`, `updateAum`, and `addReward`, but the `Vault` contract is its own ERC20 share token and never overrides the ERC20 `_update` hook nor inherits `ERC20Pausable`, so `raw transfer` and `transferFrom` remain enabled while paused. A holder can therefore move, split, or sell shares during a halt, even though the design frames pause as halting user and manager actions.

Recommendation

We recommend, if pause is intended to fully freeze the vault, overriding `_update` to revert while paused or inheriting `ERC20PausableUpgradeable`. If share transfers are intentionally allowed during pause, document this in the pause `NatSpec`.

Status: Resolved

14. Floored-integer NAV and inconsistent empty-vault NAV mislead integrators

Severity: Informational

- `src/libraries/NavMath.sol:20-25`
- `src/Vault.sol:267, 527, 752-760`

In `src/libraries/NavMath.sol:20-25`, the `navFrom` function, the NAV computed inline in `updateAum` (`src/Vault.sol:527`), and `pricePerFullShare` (`src/Vault.sol:752-760`) all express NAV per share as $\text{aum} / \text{supply}$ with floor division and no fixed-point scaling. Because the share token mirrors the asset decimals and the first deposit mints one-to-one, this quantity stays near 1 for the vault's life and discards all fractional precision, so `pricePerFullShare` and the `nav` fields in the `Deposit`, `RedemptionRequested`, and `AumUpdated` events carry little information, and are the root cause of findings 1 and 2. Separately, the empty-vault case diverges: in `src/libraries/NavMath.sol:21-22`, `navFrom` returns a nominal 1 when `supply == 0` && `aum != 0`, while in line 527, `pricePerFullShare` and the `updateAum` event both return 0.

As a result, off-chain integrators receive a low-resolution integer NAV, and the empty-vault divergence can mislead monitoring into treating a positive-AUM vault as valueless. Core share accounting is unaffected because conversions use `Math.mulDiv` on raw `aum` and `supply`.

We classify this issue as informational since it is a precision and reporting defect with no on-chain fund loss on the default path.

Recommendation

We recommend computing and storing NAV at a fixed high-precision scale (for example `Math.mulDiv(aum, SCALE, supply)` with `SCALE` of `1e18`) and using it consistently for `originalNav`, the `override math`, the event fields, and `pricePerFullShare`, and making all empty-vault paths agree on a single value.

Status: Resolved

15. `symbol_emptiness` is not validated, degrading off-chain display

Severity: Informational

- `src/Vault.sol:196-197`

In `src/Vault.sol:196`, the `initialize` function validates that `name_` is non-empty but applies no equivalent check to `symbol_`. An ERC20 token with an empty symbol is valid by the standard but unusual and may cause issues with off-chain tooling, exchanges, and wallets that rely on the symbol for display. As a result, a share token can be deployed with an empty symbol, degrading off-chain display and tooling.

Recommendation

We recommend adding `if (bytes(symbol_).length == 0) revert VaultErrors.EmptyTokenName();` (or a dedicated `EmptyTokenSymbol` error) immediately after the name check.

Status: Resolved

16. Unused custom errors increase code surface without affecting behavior

Severity: Informational

- `src/errors/VaultErrors.sol:14` (`InvalidFunds`)
- `src/errors/VaultErrors.sol:46` (`FundsManagerNotSet`)
- `src/errors/VaultErrors.sol:52` (`IncorrectPaymentAmount`)

The `VaultErrors` library declares three custom errors that are never used by the `Vault` contract:

```
error InvalidFunds(string reason);
error FundsManagerNotSet();
error IncorrectPaymentAmount(uint256 expected, uint256 received);
```

Unused errors are dead code. They do not currently affect runtime behavior, but they make the error surface larger than necessary and can confuse readers about validation paths that do not exist.

Recommendation

We recommend removing the unused custom errors, or use them in the relevant validation paths if the checks are intended to exist.

Status: Resolved

Proof of Concept

P.1 — Floored-integer override NAV underpays redeemers, causing realized loss of funds

The following Foundry tests was executed using `test/Base.t.sol` fixtures (6-decimal mock asset, ExternallyManaged deploy helpers). The test passes, confirming that applying an override at the only in-band NAV underpays the redeemer by the fractional part of the NAV.

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;

import {VaultTestBase} from "../Base.t.sol";
import {Vault} from "../src/Vault.sol";

/// Issue - the override path values shares at the floored integer NAV.
contract POC_OverrideNav is VaultTestBase {
    uint256 constant DEPOSIT = 1_000 * ASSET_UNIT; // 1e9 base units
    uint256 constant AUM_NEW = 1_020 * ASSET_UNIT; // +2% (== default band)
    uint256 constant REDEEM_SHARES = 300_000_000; // 3e8 shares

    function _setupFractionalNav() internal returns (Vault v, uint64 id, uint256 requestTimePayout) {
        v = deployExternal();
        depositAs(v, user, DEPOSIT); // aum = supply = 1e9, NAV
        vm.prank(aumUpdater);
        v.updateAum(AUM_NEW); // true NAV 1.02, floors to
        (id, requestTimePayout) = requestRedeemAs(v, user, REDEEM_SHARES);
    }

    function test_overridePathUnderpaysRedeemer() public {
        (Vault v, uint64 id, uint256 requestTimePayout) = _setupFractionalNav();
        uint256 balBefore = asset.balanceOf(user);
        approveAs(v, id, 1, true); // overrideNav = 1 (only
        uint256 received = asset.balanceOf(user) - balBefore;
        assertEq(requestTimePayout, 306_000_000); // correct full-precision
        assertEq(received, 300_000_000); // override pays shares * 1
        assertLt(received, requestTimePayout); // redeemer underpaid by 2%
    }
}
```

P.2 — Override deviation guard skipped at zero NAV allows arbitrary settlement

The following Foundry test was executed using the project's own test/Base.t.sol fixtures (6-decimal mock asset, ExternallyManaged deploy helpers). The test passes confirming that a single allowed 2% AUM downtick floors originalNav to 0 and thereby bypasses the deviation guard.

```
/// Issue – deviation guard bypassed when originalNav floors to 0.
contract POC_Bypass is VaultTestBase {
    function test_poc_deviationGuardBypassedWhenNavFloorsToZero() public {
        Vault vault = deployExternal();
        depositAs(vault, user, 2000 * ASSET_UNIT); // aum = supply = 2e9, NAV
= 1
        vm.prank(aumUpdater);
        vault.updateAum(1_960_000_000); // -2% (allowed); NAV
floors to 0
        uint256 shares = vault.balanceOf(user);
        (uint64 id, uint256 fairAssetAmount) = requestRedeemAs(vault, user,
shares);
        assertEq(vault.request(id).originalNav, 0); // precondition: floors to
0
        uint256 overrideNav = 100; // ~1_000_000 bps deviation
        uint256 payout = shares * overrideNav;
        asset.mint(redeemManager, payout);
        vm.startPrank(redeemManager);
        asset.approve(address(vault), payout);
        vault.approveRequest(id, overrideNav, true); // no revert: guard
bypassed
        vm.stopPrank();
        assertEq(asset.balanceOf(user), payout);
        assertGt(payout, fairAssetAmount * 50); // escaped the 2% bound
    }
}
```